AM

AstroManager

Logo placeholder

AstroManager

User Manual

Version **1.2.0**

Comprehensive Astrophotography File Management Suite

April 2026

Python 3.8+ / PyQt6 / Astropy

Designed for astrophotographers who demand professional-grade file management.

Contents

1 Introduction

1.1 What is AstroManager?

AstroManager is a comprehensive, professional-grade file management suite designed specifically for astrophotographers. Built with Python and PyQt6, it provides a modern, cosmic-themed interface with **15 organized tabs** covering every aspect of astrophotography data management:

- Universal file analysis (FITS, XISF, FITS.FZ)
- Multi-codec compression with 9 profiles
- Mass header editing with NINA compatibility
- Flat frame management and coverage tracking
- Frame quality analysis with PSF fitting and star detection
- WBPP-ready export with automatic calibration matching
- Target tracking over time with SIMBAD integration
- Interactive celestial sky chart
- Observation shoot calendar with heatmap
- PixInsight log analysis
- Mount tracking analysis (MountMonitor)
- Observation history and statistics dashboard
- Built-in database browser (cameras, telescopes, filters, targets)
- Disk space analysis and duplicate detection
- ASIAIR import workflow

AstroManager handles **200,000+ files** efficiently through parallel processing, multi-tier caching, and automatic hardware detection.

1.2 Supported Formats

Format	Extension	Description
FITS	`.fits`, `.fit`, `.fts`	Standard astronomical image format
XISF	`.xisf`	PixInsight native format with XML metadata
FITS.FZ	`.fits.fz`, `.fz`	RICE tile-compressed FITS (fpack)

1.3 System Requirements

Component	Requirement
Python	3.8 or newer (3.11+ recommended)
Operating System	Windows 10/11, Linux (Ubuntu 20.04+, Manjaro, etc.), macOS 12+
RAM	4 GB minimum, 8 GB+ recommended for large datasets
Storage	SSD recommended for best performance
GUI Framework	PyQt6 6.2+
Core Libraries	numpy, astropy, matplotlib, pandas, Pillow, reportlab, tqdm, requests, xisf
Optional	scipy (PSF fitting, DBSCAN), astroquery (SIMBAD), psutil (auto-tuning), zstandard, lz4

1.4 What's New in v1.2.0

Version 1.2.0 is a major feature release inspired by Athenaeum/rustafits (Vilen Sharifov). It introduces five new tabs and several cross-cutting features:

New Tabs in v1.2.0

- **Quality Analysis** — Per-frame star detection, 2D PSF Moffat/Gaussian fitting, quality scoring 0–100, satellite trail detection, interactive star maps
- **WBPP Export** — PixInsight WBPP-ready folder organization with 8-parameter calibration matching, scoring 0–1, template tokens, and preview tree
- **Sky Chart** — Aitoff/Mollweide projection of all targets, color-coded by object type, filter, or integration time, with Milky Way band overlay
- **Calendar** — Monthly grid and yearly GitHub-style heatmap of imaging sessions with statistics dashboard
- **Tags & Workflow** — Colored tags and Stage/WIP/Archive workflow stages for target classification

New Cross-Cutting Features in v1.2.0

- **DBSCAN Clustering** — Automatic target deduplication by RA/Dec coordinates using DBSCAN with Haversine distance
- **Database Schema v4** — New tables for tags, workflow stages, and frame quality cache
- **Batch Plate Solving** — Solve all unsolved raw LIGHT frames in one click, with in-place XISF header update

2 Installation

2.1 Recommended: Portable Launcher (NAS / Multi-PC)

The recommended installation method uses portable launcher scripts that automatically manage Python virtual environments. This approach works seamlessly when the project folder lives on a NAS or synced drive shared across multiple computers.

```
git clone https://github.com/ARP273-ROSE/astromanager.git
cd astromanager

# Windows
launch.bat

# Linux / macOS
chmod +x launch.sh && ./launch.sh
```

The launcher scripts perform the following steps automatically:

1. **Detect Python** on the local machine (`py -3`, `python`, `python3`).
2. **Create a virtual environment locally** — not in the project folder:
   - Windows: `%LOCALAPPDATA%\AstroManager\venv`
   - Linux/macOS: `$XDG_DATA_HOME/AstroManager/venv` (default: `~/.local/share/`)
3. **Install dependencies** from `requirements.txt` (only when they change).
4. **Launch** the application with `pythonw` (no console window on Windows).

Why local venv?

Storing the virtual environment locally rather than in the project folder prevents NAS synchronization from constantly overwriting it. Each PC gets its own native Python binaries, while the project data (database, config, analysis results) syncs normally.

Legacy Launchers

The older `run.bat` / `run.sh` scripts still work but create the venv inside the project folder. Use `launch.bat` / `launch.sh` instead for multi-PC portability.

2.2 Manual Installation

If you prefer manual installation:

```
pip install -r requirements.txt
python astromanager.py
```

AstroManager also auto-installs missing dependencies on first launch, so simply running `python astromanager.py` may be sufficient.

2.3 Standalone Executable (PyInstaller)

You can build a self-contained executable that bundles Python, all dependencies, and data files.

2.3.1 Windows (.exe)

```
pip install pyinstaller
build.bat
# Output: dist\AstroManager\AstroManager.exe
```

2.3.2 Linux / macOS

```
pip install pyinstaller
chmod +x build.sh && ./build.sh
# Output: dist/AstroManager/AstroManager
```

The build scripts use the `astromanager.spec` file to bundle the application icon, built-in databases, configuration templates, and all Python dependencies into a single distributable directory.

3 User Interface Overview

3.1 15-Tab Layout

AstroManager organizes its features into 15 tabs, listed in order from left to right:

#	Tab	Purpose
1	Analysis	Universal file analysis with SIMBAD, plate solving, weather
2	Compression	Multi-codec compression and format conversion
3	ASIAIR Import	Import workflow for ASIAIR capture sequences
4	Header Editor	Mass FITS/XISF header editing
5	Flat Manager	Flat frame grouping and coverage
6	Quality Analysis	Per-frame star detection and quality scoring (NEW)
7	WBPP Export	PixInsight-ready file export with calibration matching (NEW)
8	Target Tracking	Observation timeline and integration statistics
9	Sky Chart	Interactive celestial map of observed targets (NEW)
10	Calendar	Monthly grid and yearly heatmap (NEW)
11	PixInsight	WBPP/FBP log analysis
12	Mount Tracking	MountMonitor log analysis
13	History & Stats	Dashboard, rankings, temporal analysis
14	Database Browser	Built-in cameras, telescopes, filters, targets
15	Disk Space	Storage analysis and duplicate detection

3.2 Cosmic Dark Theme

AstroManager uses a custom "Cosmic Dark" theme optimized for use in dark observatory environments. The palette is built around deep space backgrounds (`#0a0e1a`) with muted, desaturated accents:

- **Accent cyan** (`#94b8c8`) — Primary interactive elements, links, progress bars
- **Accent purple** (`#a8a0c0`) — Secondary accents
- **Success green** (`#88b098`) — Positive indicators, good quality scores
- **Warning tan** (`#b8a880`) — Caution indicators, mediocre scores
- **Error coral** (`#b89090`) — Error indicators, poor scores

All colors are deliberately low-saturation to avoid eye strain during nighttime use.

3.3 Bilingual Support (EN/FR)

AstroManager automatically detects the system language and displays all text, tooltips, labels, error messages, and menu items in either English or French. The detection uses `locale.getdefaultlocale()` with English as the fallback.

You can override the language in **Tools** → **Settings** → **General** → **Language** (choices: `auto`, `en`, `fr`).

3.4 Console Output

A collapsible console panel at the bottom of the window shows real-time operation logs. It uses a monospace font (Consolas) and supports colored output for errors (coral), warnings (tan), and info (cyan).

Toggle the console with `Ctrl+‘` or resize the splitter.

3.5 Progress Bar

A global progress bar at the bottom displays:

- Current phase name (e.g., "Scanning files…", "Fitting PSF…")
- Percentage complete
- Estimated time remaining (ETA)

3.6 Keyboard Shortcuts

Shortcut	Action
`Ctrl+,`	Open Settings dialog
`Ctrl+Q`	Quit the application
`Ctrl+O`	Open a folder (Analysis tab)
`` Ctrl+` ``	Toggle console panel
`F1`	Open User Guide

4 Analysis Tab

The Analysis tab is the core of AstroManager. It performs recursive analysis of entire folder trees containing FITS, XISF, and FITS.FZ files, extracting comprehensive metadata and generating professional reports.

4.1 Folder Selection

Click **Browse** to select a root folder. AstroManager will recursively scan all subdirectories, excluding system folders and previously extracted duplicate folders (prefixed with `extracted_`, `duplicates_`, etc.).

4.2 Analysis Options

Option	Description
SIMBAD Resolution	Query the SIMBAD astronomical database for canonical target names and object types
Duplicate Detection	Detect duplicates using both name-based and content-based signature matching
Plate Solving	Solve astrometry using ASTAP or Astrometry.net to detect focal reducers and correct WCS
Weather Fetch	Retrieve historical weather data from Open-Meteo for each observation night
Workers	Number of parallel threads for I/O (0 = auto-detect based on CPU cores)

4.3 SIMBAD Integration

When enabled, AstroManager queries the SIMBAD database (CDS, Strasbourg) to resolve target names to their canonical designations and retrieve object types (Galaxy, Emission Nebula, Planetary Nebula, Open Cluster, etc.). Results are cached in the local SQLite database for offline access.

4.4 Plate Solving

AstroManager supports two plate solving backends:

- **ASTAP** (Han Kleijn) — Fast local solver, recommended
- **Astrometry.net** — Online or local solver

Plate solving automatically detects focal reducers (0.67x, 0.72x, 0.8x) by comparing the solved focal length to the expected value from the telescope database. The WCS solution and corrected focal length are written back to the file headers.

Batch solving: AstroManager can solve all unsolved raw LIGHT frames in one click. Large images are automatically binned for faster solving. XISF headers are updated in-place (modifying only the ∼8 KB header block, not the full 60 MB image data). PixInsight processed files, masters, and calibration frames (Dark/Flat/Bias) are automatically skipped.

4.5 Weather Integration

Historical weather data is fetched from the Open-Meteo API (free, no API key required). Data includes:

- Temperature, humidity, dew point
- Cloud cover percentage
- Precipitation and wind speed
- Atmospheric pressure

Results are cached in a 365-day SQLite cache for offline access.

4.6 Report Generation

AstroManager generates reports in multiple formats:

Format	Description
LaTeX/PDF	Professional report with tables, charts, and statistics (requires pdflatex)
HTML	Standalone HTML file with embedded CSS (always available)
CSV	Spreadsheet-compatible data export
AstroBin CSV	Specialized CSV with AstroBin filter ID mapping (Ha=4663, OIII=4752, SII=4844, L=2906, R/G/B, NII)

Reports include per-night, per-target, and per-filter statistics, equipment identification from built-in databases, mosaic panel detection, and duplicate analysis.

5 Compression Tab

5.1 Overview

The Compression tab provides bidirectional format conversion between FITS, XISF, and FITS.FZ with 9 compression profiles. All output is fully compatible with PixInsight.

5.2 Compression Profiles

Profile	Ratio	Speed	Best For
`zlib_1`	~30%	Fast	Temporary files
`zlib_6`	~50%	Medium	**General archival (recommended)**
`zlib_9`	~55%	Slow	Maximum zlib compression
`zstd_3`	~45%	Fast	Quick archival
`zstd_6`	~55%	Medium	Better than zlib_6
`zstd_10`	~60%	Slow	High compression
`zstd_19`	~65%	Very slow	Long-term archival
`lz4`	~25%	Ultra-fast	Working files, fast iteration
`lz4_hc`	~35%	Fast	Better LZ4 compression

Choosing a Profile

For most astrophotographers, `zlib_6` offers the best balance of compression ratio and speed. Use `lz4` when you need to quickly compress working files and plan to recompress later. Use `zstd_19` for permanent deep archival where speed is not a concern.

5.3 Format Conversion

Supported conversion paths:

- **FITS** → **XISF**: Converts to PixInsight native format with chosen compression
- **XISF** → **FITS**: Decompresses back to standard FITS
- **FITS** → **FITS.FZ**: RICE tile compression via Astropy (no external fpack needed)
- **FITS.FZ** → **FITS**: Decompresses tile-compressed files

5.4 SHA-256 Integrity Verification

When enabled, AstroManager computes SHA-256 checksums before and after conversion to verify data integrity. Any mismatch triggers an error and the output file is discarded.

5.5 Parallel Processing

Compression uses `ProcessPoolExecutor` for CPU-bound work, automatically scaling to the number of physical CPU cores. You can override the worker count in Settings.

Disk Space

Ensure you have sufficient disk space for the output files. Compression creates new files without deleting the originals (unless "Delete source after conversion" is enabled in Settings).

6 ASIAIR Import Tab

6.1 Overview

The ASIAIR Import tab provides a dedicated workflow for importing capture sequences from ZWO ASIAIR controllers. ASIAIR organizes files differently from NINA or SGPro, so this tab handles the translation.

6.2 Import Workflow

1. **Select ASIAIR capture folder** — Browse to the root of your ASIAIR output (typically organized as `autorun/` or `plan/`).
2. **Configure target settings** — Set the target name, organization preferences, and output folder.
3. **Import and organize** — AstroManager copies and reorganizes files by target, filter, and date.

6.3 Header Fixes

ASIAIR cameras (ZWO ASI series) produce FITS files with BZERO/BSCALE/BLANK keywords for unsigned integer storage (per the FITS standard). AstroManager automatically handles these keywords during import, ensuring compatibility with PixInsight, Siril, and other processing software.

> **Note**
>
> The BZERO/BSCALE handling was a critical fix in v1.1.5. Files are read with `memmap=False` to avoid memory-mapping issues with these keywords.

7 Header Editor Tab

7.1 Overview

The Header Editor allows mass editing of FITS and XISF headers across hundreds or thousands of files simultaneously. It is organized by category for easy navigation.

7.2 Categories

Category	Keywords
Acquisition	EXPTIME, DATE-OBS, IMAGETYP, FRAME, GAIN, OFFSET
Camera	INSTRUME, CCD-TEMP, XPIXSZ, YPIXSZ, XBINNING, YBINNING
Filter	FILTER
Telescope	TELESCOP, FOCALLEN, APTDIA, SITEOBS
Observer	OBSERVER, SITENAME, SITELAT, SITELONG, SITEELEV
Object	OBJECT, RA, DEC, EQUINOX
Software	SWCREATE, PROGRAM

7.3 NINA Pattern Builder

AstroManager supports NINA-style filename patterns using tokens:

Token	Expands To
`$IMAGETYPE$`	LIGHT, DARK, FLAT, BIAS
`$TARGETNAME$`	Object name from OBJECT keyword
`$FILTER$`	Filter name
`$EXPOSURE$`	Exposure time in seconds
`$DATE$`	Date from DATE-OBS
`$CAMERA$`	Camera name from INSTRUME
`$TELESCOPE$`	Telescope name from TELESCOP
`$GAIN$`	Gain value
`$TEMP$`	Sensor temperature
`$BINNING$`	Binning (e.g., 1x1)

7.4 Auto-Rename After Edit

After applying header changes, AstroManager automatically proposes renaming files to match the NINA pattern configured in Settings. This ensures filenames stay synchronized with their header metadata.

7.5 Undo

All header edits support undo. AstroManager stores the original values before modification and can revert changes if needed.

Warning

While header editing supports FITS, XISF, and FITS.FZ formats, XISF editing uses `defusedxml` for XML security. Always verify critical files after editing.

8 Flat Manager Tab

8.1 Overview

The Flat Manager helps organize, analyze, and track flat calibration frames. Proper flat field correction is essential for removing vignetting, dust shadows, and optical path artifacts.

8.2 Automatic Grouping

Flat frames are automatically grouped by:

- Date of acquisition
- Optical setup (telescope + focal length)
- Filter name
- Binning mode
- Rotation angle
- Sensor temperature

8.3 Coverage Report

For each group, the Flat Manager reports:

- **Complete** — Sufficient flats for reliable master creation (typically 20+)
- **Insufficient** — Some flats exist but below recommended count

- Missing — No flats found for this configuration

8.4 Master Flat Tracking

The Flat Manager tracks which master flats have been created and links them to target light frames based on matching criteria (setup, filter, binning, date proximity).

9 Quality Analysis Tab

New in v1.2.0

This tab is new in v1.2.0, inspired by Athenaeum/rustafits's PSF analysis pipeline.

9.1 Overview

The Quality Analysis tab performs automated per-frame quality assessment using star detection and Point Spread Function (PSF) fitting. It produces a quality score from 0 to 100 for each frame, enabling data-driven frame rejection decisions.

9.2 Star Detection

AstroManager uses **scipy peak detection** to identify stars in each frame:

1. The image is loaded and background-subtracted.
2. Local maxima above a configurable threshold (typically 5σ above background) are identified.
3. Candidate peaks are filtered by size, brightness, and edge proximity.
4. The brightest N stars (configurable, default 200) are selected for PSF fitting.

9.3 PSF Fitting

For each detected star, AstroManager fits a 2D elliptical PSF model:

- **Primary model: Moffat function** — More accurate for astronomical seeing, with a power-law wing profile:

$$I(r) = I_0 \left(1 + \left(\frac{r}{\alpha}\right)^2\right)^{-\beta}$$

- **Fallback: Gaussian function** — Used when Moffat fitting fails to converge:

$$I(r) = I_0 \exp\left(-\frac{r^2}{2\sigma^2}\right)$$

Both models are fitted as 2D elliptical functions to capture asymmetric star profiles caused by tracking errors, optical aberrations, or atmospheric effects.

9.4 Per-Frame Metrics

Metric	Description
FWHM	Full Width at Half Maximum (arcseconds or pixels). Measures star sharpness. Lower is better.
HFR	Half-Flux Radius. Alternative sharpness measure used by autofocus systems.
Eccentricity	Ellipticity of star profiles (0 = perfect circle, 1 = line). Measures tracking/guiding quality.
SNR	Signal-to-Noise Ratio of detected stars. Higher is better.
Star Count	Number of successfully fitted stars. Very low counts indicate problems.
Background	Mean background level and noise (standard deviation).
Roundness	Average roundness of star profiles (complement of eccentricity).

9.5 Quality Score (0–100)

Each frame receives a composite quality score computed from weighted metrics:

Metric	Weight
FWHM (sharpness)	35%
Eccentricity (roundness)	20%
SNR (signal quality)	20%
Star Count (detection)	15%
Roundness (symmetry)	10%

Scores are color-coded in the results table:

- **80–100**: Excellent quality
- **60–79**: Good quality
- **40–59**: Mediocre quality
- **0–39**: Poor quality — consider rejection

9.6 Auto-Reject Suggestions

AstroManager can automatically suggest frames for rejection based on:

- **Absolute thresholds**: Configurable limits for FWHM, eccentricity, SNR, and star count
- **Batch-relative outliers**: Frames that deviate significantly from the batch median (e.g., FWHM $> 2\sigma$ above median)

9.7 Satellite Trail Detection

AstroManager detects satellite trails using a **Rayleigh R^2 test** for directional coherence:

1. The elongation direction (position angle) of each detected star is measured.
2. A Rayleigh test checks whether elongation directions are randomly distributed or aligned along a common axis.
3. Strong directional coherence (high R^2) indicates a satellite or aircraft trail contaminating the frame.

9.8 Star Map Visualization

Double-click any frame in the results table to open the **Star Map** dialog. This interactive visualization shows:

- Each detected star's position on the frame
- PSF ellipse shape and orientation (elongation direction)
- Color coding by individual star quality
- Summary statistics overlaid on the image

9.9 Export

- **Accepted file list** (`.txt`): List of filenames that passed quality thresholds, ready for import into PixInsight or Siril.
- **Full metrics report** (`.csv`): Complete per-frame statistics including all metrics, scores, and rejection flags.

10 WBPP Export Tab

New in v1.2.0

This tab is new in v1.2.0, inspired by Athenaeum/rustafits's WBPP-ready export with calibration tree visualization.

10.1 Overview

The WBPP Export tab organizes your FITS/XISF files into a folder structure ready for PixInsight's **Weighted Batch PreProcessing (WBPP)** script. It automatically matches calibration frames (darks, flats, biases) to light frames and provides a preview of the planned structure before exporting.

10.2 Template Tokens

The output folder structure is defined using template tokens:

Token	Expands To
`{OBJECT}`	Target name from OBJECT keyword
`{IMAGETYP}`	Frame type: LIGHT, DARK, FLAT, BIAS
`{FILTER}`	Filter name
`{DATE}`	Observation date
`{CAMERA}`	Camera identifier
`{TELESCOPE}`	Telescope identifier
`{EXPTIME}`	Exposure time
`{TEMP}`	Sensor temperature
`{GAIN}`	Gain value
`{BINNING}`	Binning mode

The default template produces: `{OBJECT}/{IMAGETYP}/{FILTER}/` for lights and `{OBJECT}/DARK/{EXPTIME}s_{` for darks.

10.3 Calibration Auto-Matching

AstroManager matches calibration frames to lights using an 8-parameter scoring system:

Parameter	Exact	Warning	Ignore
Camera (INSTRUME)	Required	—	—
Temperature (±2°C)	Match	±5°C	Skip
Filter (FILTER)	Match	Similar	Skip
Exposure (EXPTIME)	Match	±20%	Skip
Binning (XBINNING)	Match	—	Skip
Gain (GAIN)	Match	±10%	Skip
Date proximity	Same week	Same month	Skip
Image dimensions	Match	—	Skip

10.4 Calibration Scoring (0–1)

Each calibration match receives a quality score from 0.0 (worst) to 1.0 (perfect). The scoring considers:

- **Parameter match quality**: Exact matches score higher than approximate matches.
- **Fallback chains**: If no matching darks are found, AstroManager tries DarkFlat → Dark → Bias (in order of preference).
- **Three matching modes per parameter**: Exact (strict), Warning (loose), and Ignore (skip the parameter entirely).

10.5 Preview Tree

Before exporting, the preview tree shows the planned folder structure as an interactive tree widget. Each folder node displays:

- Number of files
- Total size
- Calibration match quality (color-coded green/yellow/red)
- Warnings for missing or poorly matched calibrations

10.6 Validation Warnings

AstroManager validates the export plan and warns about:

- Missing calibration frames (no darks, flats, or biases)
- Temperature mismatches beyond the tolerance threshold
- Incomplete filter groups
- Mismatched binning between lights and calibrations

10.7 Export Modes

Mode	Description
Copy	Physically copy files to the destination (safest, uses most disk space)
Symlink	Create symbolic links (saves disk space, requires OS support)
Preview-only	Show the planned structure without writing any files

Tip

Use **Preview-only** mode first to verify the structure and calibration matching before committing to a full copy.

11 Target Tracking Tab

11.1 Overview

The Target Tracking tab provides a historical view of your observations for each target. It combines data from the analysis database with SIMBAD lookups and weather forecasts to give a complete picture of your imaging history.

11.2 Observation Timeline

A bar chart displays the integration time accumulated for each observation session, organized chronologically. Bars are color-coded by filter (e.g., blue for Luminance, red for H-alpha, teal for OIII).

11.3 Integration Statistics

Per-target statistics include:

- Total integration time (hours) across all sessions
- Integration per filter (with bar chart)
- Number of sessions, frames, and unique filters
- Best session identification by HFR, FWHM, and weather quality
- Equipment usage (telescopes, cameras, mounts)

11.4 Weather Forecast

The Target Tracking tab displays a multi-day weather forecast for upcoming nights, helping you plan future observation sessions. Data comes from the Open-Meteo API using your observatory coordinates (configured in Settings).

11.5 Target Visibility

Altitude charts show when a target is observable from your location, taking into account:

- Target declination and right ascension
- Observatory latitude, longitude, and elevation
- Twilight boundaries (civil, nautical, astronomical)
- Meridian transit time

12 Sky Chart Tab

New in v1.2.0

This tab is new in v1.2.0, inspired by Athenaeum/rustafits's interactive sky chart with d3-celestial.

12.1 Overview

The Sky Chart tab displays an interactive celestial map of all your observed targets using an Aitoff or Mollweide all-sky projection. It provides a visual overview of your imaging coverage across the sky.

12.2 Projection Modes

- **Aitoff projection**: Equal-area projection that preserves area relationships. Good for seeing the overall distribution of targets.
- **Mollweide projection**: Another equal-area projection with a more elliptical shape. Often used in astronomical catalogs.

Both projections display the full celestial sphere with coordinate grid lines (Right Ascension and Declination).

12.3 Color Modes

Switch between three color-coding schemes:

Mode	Description
Object Type	Galaxies (rose), Emission nebulae (green), Planetary nebulae (lavender), Clusters (gold), Dark nebulae (grey), etc.
Filter	Colors match your dominant filter: H-alpha (red), OIII (teal), SII (orange), Luminance (blue), RGB (mixed)
Integration Time	Gradient from dim (low integration) to bright (high integration)

12.4 Target Size

Each target is drawn as a circle whose size is proportional to the total integration time accumulated. Targets with many hours of data appear larger, making it easy to spot your most-invested objects.

12.5 Milky Way Band

A semi-transparent overlay approximates the Milky Way's galactic plane. This is toggleable and helps visualize the relationship between your targets and the galaxy.

12.6 Interactive Features

- **Hover tooltips**: Move your mouse over any target to see its name, object type, RA/Dec coordinates, and total integration time.
- **Click to select**: Clicking a target emits a signal for cross-tab navigation (e.g., jump to the Target Tracking tab for that object).
- **Toggle labels**: Show or hide target name labels on the chart.
- **Toggle grid**: Show or hide the RA/Dec coordinate grid.

12.7 Export

Export the sky chart as a high-resolution PNG image (200 DPI) for inclusion in presentations, reports, or social media posts.

13 Calendar Tab

New in v1.2.0

This tab is new in v1.2.0, inspired by Athenaeum/rustafits's shoot calendar.

13.1 Overview

The Calendar tab provides a visual overview of your imaging activity over time, combining a monthly calendar grid with a yearly heatmap and statistics dashboard.

13.2 Monthly View

The monthly view displays a standard calendar grid where each day cell shows:

- Number of frames captured
- Total integration time
- Target names observed
- Color intensity reflecting activity level

Navigate between months using arrow buttons. Days with imaging sessions are highlighted; click any day to see detailed session information.

13.3 Yearly Heatmap

The yearly heatmap displays a GitHub-style contribution map: 52 weeks $\times$ 7 days, where each cell's color intensity reflects the amount of imaging activity on that night. The color scale ranges from the dark background color (no data) through increasingly bright shades of the accent cyan:

Level	Color	Meaning
None	Dark (`#0a0e1a`)	No imaging
Low	`#1a2a3a`	Brief session
Medium	`#4a7a8a`	Moderate session
High	`#6a9aaa`	Long session
Very High	`#94b8c8`	Extensive session

13.4 Statistics Dashboard

Six stat cards display aggregate metrics:

- **Imaging Nights**: Total number of nights with at least one frame
- **Total Integration**: Cumulative integration time across all sessions
- **Best Month**: The month with the most imaging activity
- **Avg Frames/Night**: Average number of frames per imaging session
- **Longest Streak**: Maximum consecutive imaging nights
- **Unique Targets**: Number of distinct objects observed

13.5 Session Details

Click any day in the calendar or heatmap to see a detailed breakdown:

- Per-target breakdown: filter, frame count, integration time
- HFR and FWHM averages
- Telescope and camera used
- Weather conditions (if available)

13.6 Export

Export the calendar view as a PNG image for your records or sharing.

14 PixInsight Tab

14.1 Overview

The PixInsight tab imports and analyzes processing logs from PixInsight's WBPP (Weighted Batch PreProcessing) and FBP (Fast Batch Preprocessing) scripts.

14.2 Log Analysis

After importing a log file, AstroManager extracts:

- Per-frame quality metrics: FWHM, SNR, eccentricity, weight
- Rejection statistics: percentage of frames rejected per filter
- Integration results: total accepted frames, effective integration time
- Calibration tracking: which master darks, flats, and biases were applied

14.3 Per-Frame Statistics

A detailed table shows each subframe with its quality metrics. Frames are color-coded by quality, and rejected frames are clearly marked. This helps identify which frames were dropped and why.

15 Mount Tracking Tab

15.1 Overview

The Mount Tracking tab analyzes log files from **MountMonitor**, a mount tracking quality assessment tool. It provides deviation charts, FFT periodic error analysis, and per-target quality grades.

15.2 Supported Log Formats

Extension	Content
`.dat`	Main tracking data (RA/DEC deviations)
`.dti`	Time synchronization data
`.fft`	FFT periodic error analysis
`.env`	Environment data (temperature, pressure)
`.log`	General log entries

15.3 Tracking Quality Dashboard

The dashboard displays:

- **RA RMS**: Root Mean Square deviation in Right Ascension (arcseconds)
- **DEC RMS**: Root Mean Square deviation in Declination (arcseconds)
- **Tracking %**: Percentage of time in TRACKING state (vs. SLEWING/PARKED/IDLE)
- **Quality Grade**: A through F rating based on combined RMS

RA deviations are corrected by $\cos(\delta)$ to reflect true arcseconds on the sky (where δ is the declination).

15.4 Target Segmentation

AstroManager automatically detects pointing changes by analyzing jumps in RA/DEC values. Each pointing segment is treated as a separate target observation, with independent statistics.

15.5 Deviation Timeline

An interactive chart shows RA and DEC deviations over time. Only TRACKING samples are plotted (SLEWING, PARKED, and IDLE states are excluded).

15.6 FFT Periodic Error Analysis

Periodic error peaks are identified using Fast Fourier Transform:

- Dominant PE periods and amplitudes for each axis
- Worm gear frequency detection (typically 8–10 minute period for common mounts)
- Harmonic identification

15.7 Per-Target Statistics

A table lists each detected target segment with:

- RMS deviations (RA and DEC)
- Peak-to-peak range
- Duration
- Quality grade

16 History & Statistics Tab

16.1 Overview

The History & Statistics tab provides a comprehensive dashboard of your entire observation history stored in the SQLite database.

16.2 Dashboard

Six stat cards provide an at-a-glance summary:

- Total targets observed
- Cumulative integration time
- Total frames captured
- Number of imaging nights
- Average HFR across all sessions
- Number of telescopes used

16.3 Target Rankings

Targets are ranked by total integration time, with per-target breakdowns showing:

- Filter distribution (bar chart)
- Number of sessions
- Best session quality

16.4 Per-Filter Statistics

Usage statistics for each filter across all targets:

- Number of targets using this filter
- Total sessions and frames
- Average HFR and FWHM
- Total integration time

16.5 Equipment Statistics

Aggregated statistics per:

- Telescope (frames, integration, targets, nights)
- Camera (same breakdown)
- Setup combination (telescope + camera pair)

16.6 Temporal Analysis

- **Monthly**: Activity trend over months
- **Yearly**: Year-over-year comparison
- **Day of week**: Which days produce the most imaging (usually clear weeknights!)

16.7 Best Nights

Nights ranked by:

- **Quality**: Best average HFR (sharpest seeing)
- **Productivity**: Most integration time accumulated

16.8 Object Type Statistics

Integration time broken down by astronomical object type:

- Galaxies, Emission nebulae, Planetary nebulae
- Open clusters, Globular clusters
- Dark nebulae, Reflection nebulae
- Other types

16.9 Export and Import

Format	Description
JSON export	Complete history dump including all observations, targets, weather, and metadata
CSV export	Spreadsheet-compatible export of observations
JSON import	Restore a previously exported history

17 Database Browser Tab

17.1 Overview

The Database Browser tab provides read-only access to AstroManager's built-in reference databases. These databases are used internally for automatic identification of cameras, telescopes, filters, and astronomical targets.

17.2 Built-in Databases

Database	Entries	Coverage
Cameras	1,614 sensors + 2,448 mappings	ZWO, QHY, Atik, FLI, SBIG, Player One, Touptek, Moravian, Canon, Nikon, Sony, and more
Telescopes	3,991 models + 6,605 mappings	Takahashi, Celestron, Sky-Watcher, Meade, APM, William Optics, PlaneWave, Officina Stellare, TEC, RCOS, and more
Filters	1,612 filters + 701 aliases	Baader, Astronomik, Chroma, ZWO, Optolong, Antlia, IDAS, Astrodon, Custom Scientific
Targets	32,923 objects	Messier (110), NGC/IC (32,411), Arp (338), Solar System (64), plus Sharpless, Barnard, Caldwell, RCW, LBN, PGC, Abell, LDN, Hickson, UGC

Total: approximately 41,000 reference entries.

17.3 Search and Filter

Each database tab supports:

- Free-text search by name
- Filter by brand, type, or catalog
- Sorting by any column

18 Disk Space Tab

18.1 Overview

The Disk Space tab analyzes your storage usage by file format, identifies duplicates, and recommends optimization strategies.

18.2 Storage Breakdown

A summary shows total space used by each format:

- FITS files (uncompressed)
- XISF files (compressed)
- FITS.FZ files (RICE tile-compressed)
- Duplicates (name-based and content-based)

18.3 Duplicate Detection

AstroManager uses a two-level duplicate detection system:

1. **Phase 1 — Name-based**: Groups files by base name (without extension). When the same image exists in multiple formats (e.g., `NGC7000_L_001.fits` and `NGC7000_L_001.xisf`), the preferred format is kept (.xisf > .fits > .fits.fz).
2. **Phase 2 — Content-based**: Groups files by a 10-field signature tuple:
   - DATE-OBS, image type, exposure time, dimensions (NAXIS1/NAXIS2)
   - Binning, filter, object, instrument, telescope, frame number

Content-Based Signature

Both INSTRUMENT and TELESCOPE are included in the signature (added in v1.1.12) to prevent false duplicates between different hardware setups. DATE-OBS is required — files without a valid timestamp are treated as unique.

18.4 Optimization Recommendations

AstroManager suggests:

- Compressing uncompressed FITS to XISF (estimated savings)
- Removing duplicate files (estimated savings)
- Archiving old data to slower storage
- Organizing files by type, date, or target

18.5 File Organization

Built-in organization presets:

- By image type (LIGHT, DARK, FLAT, BIAS)
- By date (year/month/day)
- By target name
- Custom patterns

18.6 Storage Type Detection

AstroManager detects whether your storage is SSD or HDD and adjusts I/O strategies accordingly:

- Windows: WMI queries
- Linux: `/sys/block` inspection
- macOS: `diskutil` commands

19 Configuration

19.1 Settings Dialog

Access the Settings dialog via **Tools** → **Settings** or **Ctrl+,**.

Section	Settings
General	Language (auto/en/fr), Check for updates on startup
Observatory	Latitude, Longitude, Elevation, Timezone
Performance	Workers (0 = auto-detect), Batch size
Compression	Default profile, Delete source after conversion, Verify integrity
Analysis	SIMBAD, Plate solving, Weather defaults
Bug Reporting	Enable/disable anonymous crash reports

19.2 Configuration File

Settings are stored in `config.yaml` in the project directory:

```
application:
  language: "auto"    # auto, en, fr
  theme: "cosmic_dark"
```

```
system:
  workers: 0          # 0 = auto-detect
  batch_size: 1000

observatory:
  latitude: 51.4769   # Update for your location!
  longitude: -0.0005
  elevation_m: 46
  timezone: "UTC"

analysis:
  enable_simbad: true
  enable_plate_solving: false
  enable_weather_fetch: false

compression:
  default_profile: "zlib_6"
  verify_integrity: true

plate_solving:
  solver: "astap"     # astap or astrometry

weather:
  api_provider: "open-meteo"
  cache_duration_days: 365

bug_reporting:
  enabled: true
```

Observatory Coordinates

Make sure to update the observatory coordinates to match your actual location. These are used for weather forecasts, target visibility calculations, and altitude charts.

19.3 Command-Line Options

AstroManager can be used from the command line for scripted analysis:

```
python astromanager.py                                  # Launch GUI
python astromanager.py --folder /path/to/fits           # CLI analysis
python astromanager.py --folder /path --resolve-simbad  # With SIMBAD
python astromanager.py --folder /path --export-csv      # Export CSV
python astromanager.py --folder /path --workers 8       # 8 workers
python astromanager.py --optimize-storage /backup       # Optimize storage
python astromanager.py --version                        # Show version
python astromanager.py --reset-config                   # Reset config
python astromanager.py --vacuum-db                      # Optimize database
python astromanager.py --debug                          # Debug logging
```

20 SQLite Database

20.1 Overview

AstroManager uses an SQLite database in **WAL mode** (Write-Ahead Logging) for concurrent read access. The database file (`astromanager.db`) is stored in the project directory and syncs across PCs via NAS.

Single-PC Access

SQLite WAL mode does not support simultaneous access from multiple computers over a network filesystem. Only run AstroManager on one PC at a time.

20.2 Schema Versions

The database uses an automatic migration system. Each schema version adds new tables:

Version	Tables Added
v1	`targets`, `observations`, `weather_cache`, `header_cache`, `analysis_results`
v2	`pixinsight_sessions`, `pixinsight_subframes`, `pixinsight_integrations`, `pixinsight_frame_weights`, `pixinsight_calibrations`
v3	`mount_sessions`, `mount_tracking_data`, `mount_time_data`, `mount_fft_data`, `mount_environment_data`
v4	`tags`, `target_tags`, `observation_tags`, `frame_quality` + `workflow_stage` column on `targets`

20.3 Thread Safety

Database access is thread-safe through:

- Thread-local connections via `get_connection()` context manager
- WAL mode with 30-second busy timeout
- Nested transaction depth tracking for commit/rollback
- WAL checkpoint with TRUNCATE at application exit

20.4 Maintenance

- **Vacuum**: Run `python astromanager.py -vacuum-db` to reclaim space and optimize the database.
- **Backup**: The database is automatically backed up on application exit.
- **Reset**: Delete `astromanager.db` to recreate from scratch (all history will be lost).

21 DBSCAN Clustering & Tags

New in v1.2.0

These features are new in v1.2.0, inspired by Athenaeum/rustafits's DBSCAN clustering and tags system.

21.1 Target Clustering by RA/Dec

Over time, the same target may appear under different names or with slightly different coordinates. DBSCAN clustering helps identify and merge these duplicates.

21.1.1 How It Works

1. **Coordinate extraction**: All targets with valid RA/Dec coordinates are loaded from the database.
2. **DBSCAN clustering**: The algorithm groups targets using **Haversine angular distance** on the celestial sphere. Two configurable parameters:
   - **Epsilon** (ε): Maximum angular separation to consider targets as neighbors (default: 15 arcminutes)
   - **Min samples**: Minimum number of targets to form a cluster (default: 2)
3. **Merge suggestions**: Clusters are classified as:
   - **Strong match** ($<$ 5 arcmin): Very likely the same object
   - **Moderate match** ($<$ 15 arcmin with name similarity): Probably the same object, manual verification recommended

21.1.2 Safe Merging

Merging operates in **dry-run mode by default**:

1. Preview which targets would be merged and what the canonical name would be
2. Confirm the merge
3. Observations are moved to the canonical target, statistics are recalculated
4. The duplicate target entry is removed

21.1.3 Nearby Target Search

Search for all targets within a given angular radius of a specified position. Useful for:

- Finding objects near a target of interest
- Checking if a "new" target already exists under a different name
- Planning mosaic sessions by finding nearby objects

21.2 Tags System

21.2.1 Colored Tags

Create custom colored tags to classify your targets and observations. Examples:

- "Priority" (red), "Narrowband" (teal), "Mosaic" (purple)
- "Needs More Data" (orange), "Processing Done" (green)

Tags are stored in the database with many-to-many relationships: each target or observation can have multiple tags, and each tag can be applied to multiple targets.

21.2.2 Workflow Stages

Three predefined workflow stages help track the processing status of each target:

Stage	**Meaning**
Stage	New data, not yet started processing
WIP (Work In Progress)	Currently being processed in PixInsight/Siril
Archive	Processing complete, final result archived

Workflow stages are stored as a column on the `targets` table and persist across sessions.

22 NAS Portability

22.1 Architecture

AstroManager is designed to run from a project folder stored on a NAS (Synology, QNAP, etc.) or any cloud-synced directory (OneDrive, Dropbox, Google Drive). The key principle is:

- **Project data** (database, config, analysis results) stays in the project folder and syncs across PCs.
- **Python virtual environment** is stored locally on each PC (never in the project folder).

22.2 Local Virtual Environment

The launcher scripts store the venv outside the project directory:

OS	Venv Location
Windows	`%LOCALAPPDATA%\AstroManager\venv`
Linux	`$XDG_DATA_HOME/AstroManager/venv`
macOS	`~/.local/share/AstroManager/venv`

This prevents NAS synchronization from constantly overwriting platform-specific Python binaries. Each PC gets its own native venv, while shared data syncs normally.

22.3 Launcher Scripts

`launch.bat` (Windows):

- Detects Python via `py -3` → `python` → `python3`
- Creates venv in `%LOCALAPPDATA%\AstroManager\venv`
- Uses a `.deps_installed` marker (copy of requirements.txt) to detect when dependencies need reinstalling
- Launches via `pythonw.exe` with `start ""` (no console window)
- Validates the venv works (`python -c "import sys"`) to detect broken venvs from uninstalled Python versions

`launch.sh` (Linux/macOS):

- Detects Python via `python3` → `python`
- Creates venv in `$XDG_DATA_HOME/AstroManager/venv`
- Same marker and validation logic as the Windows version

22.4 Desktop Shortcuts

Use **Help** → **Create Desktop Shortcut** to create a desktop shortcut that targets the launcher script (not the Python executable directly). This ensures the shortcut works regardless of which drive letter the NAS is mapped to.

On Windows, the application icon is copied locally to `%APPDATA%\AstroManager\logo.ico` because Windows `.lnk` shortcuts cannot display icons from network paths.

Tip

If you move the project folder to a different location or drive, simply re-run **Help** → **Create Desktop Shortcut** to update the shortcut path.

23 Troubleshooting & FAQ

23.1 No FITS Files Found

- Verify the folder path and check that files have recognized extensions: `.fits`, `.fit`, `.fts`, `.fz`, `.xisf`
- Check file read permissions
- Ensure the folder is not excluded (folders prefixed with `extracted_`, `duplicates_`, etc. are skipped)

23.2 Database Locked

- Close all other AstroManager instances
- SQLite WAL mode does not support concurrent network access — ensure only one PC is running AstroManager at a time
- As a last resort, delete `astromanager.db` (it will be recreated, but all history will be lost)

23.3 Slow Performance

- **Check storage type**: HDD is significantly slower than SSD for scanning large folder trees
- **Reduce workers**: If RAM is limited, lower the worker count in Settings
- **Disable optional features**: Turn off SIMBAD and weather for faster analysis
- **Clear old cache**: Use **Tools** → **Clear Old Cache** to remove stale cache entries
- **Vacuum database**: Run `python astromanager.py -vacuum-db` to optimize the database

23.4 LaTeX Report Compilation Fails

- AstroManager always generates an HTML report as a fallback
- A ReportLab PDF is also generated automatically when LaTeX is unavailable
- Ensure `pdflatex` is in your system PATH if you want LaTeX PDF reports

23.5 Missing Dependencies

- AstroManager auto-installs missing packages on first launch
- For manual installation: `pip install -r requirements.txt`
- If using a system Python with restricted permissions, try: `pip install -user -r requirements.txt`

23.6 XISF Header Editing Errors

- XISF files use XML metadata. AstroManager uses `defusedxml` for secure parsing
- If you encounter namespace corruption, update to v1.1.8+ which fixed this issue
- Always back up files before bulk header editing

23.7 ASIAIR Files Fail to Read

- ASIAIR cameras produce FITS with BZERO/BSCALE keywords
- This was fixed in v1.1.5 — ensure you are running v1.1.5 or newer

- If the issue persists, try importing through the ASIAIR Import tab which applies additional header fixes

23.8 Plate Solving Fails

- Ensure ASTAP is installed and its path is in your system PATH, or configure the path in Settings
- Check that the ASTAP star database is downloaded (G17 or H17 catalog)
- For very wide-field images, you may need to adjust the search radius
- Large images are automatically binned for faster solving

23.9 Update Checker Not Working

- The update checker requires internet access to reach the GitHub API
- It uses a 5-second timeout and fails silently if the network is unavailable
- Checks are throttled to once every 24 hours (configurable)
- You can skip a specific version via the "Skip This Version" button in the update dialog
- Manual check: **Help → Check for Updates**

23.10 Broken Virtual Environment

If you see "did not find executable" errors:

- This typically happens when a Windows Store Python installation is removed
- The launcher scripts (`launch.bat`/`launch.sh`) automatically detect and recreate broken venvs
- Alternatively, delete the venv folder manually and re-launch

23.11 Crash Reports

AstroManager includes an anonymous crash reporting system:

- Crashes are automatically captured via a custom `sys.excepthook`
- Crash reports are saved as JSON files in the `crash_reports/` folder
- On restart, you are prompted to send the report (via GitHub Issues)
- All paths are anonymized (usernames replaced with `~`)
- No telemetry is sent without your explicit consent

24 Acknowledgments

AstroManager v1.2.0 would not be possible without the following projects and communities:

24.1 Inspiration

- **Athenaeum / rustafits** (Vilen Sharifov) — Major inspiration for v1.2.0 features: frame quality analysis with PSF pipeline, WBPP export with calibration tree visualization, interactive sky chart, shoot calendar, calibration scoring system, DBSCAN target clustering, and tags with workflow stages.

24.2 Core Libraries

- **Astropy** — FITS handling, RICE compression, coordinate transforms, and time utilities
- **PyQt6** — Cross-platform GUI framework

- **NumPy** / **SciPy** — Scientific computing, PSF fitting, DBSCAN clustering
- **Matplotlib** — Charts, sky projections, and visualizations
- **Pandas** — Data manipulation and analysis
- **Pillow** — Image processing
- **ReportLab** — PDF report generation

24.3 Astronomical Services

- **SIMBAD** (CDS, Strasbourg) — Astronomical database for target identification and object classification
- **Open-Meteo** — Free weather API for historical and forecast data (no API key required)
- **ASTAP** (Han Kleijn) — Fast local plate solving engine
- **Astrometry.net** — Plate solving engine
- **PixInsight** — XISF format specification, WBPP/FBP processing scripts

24.4 Community

- **CloudyNights** community — Testing, feedback, and feature suggestions
- All astrophotographers who have submitted bug reports and feature requests

---

AstroManager v1.2.0

Comprehensive Astrophotography File Management Suite

Clear skies!